

Day 4 - Functions: Args, Scope, Closures, First-Class Functions

Day 4 — Functions: Args, Scope, Closures, First-Class Functions

Objectives

- Understand what a function actually is and why they're one of the most important tools in programming
- Write functions that accept required parameters, optional parameters with defaults, and flexible numbers of arguments
- Understand "scope" — which parts of your program can see which variables, and why
- Understand closures — functions that remember values from where they were created
- Understand that functions themselves are just another kind of value in Python

What is a function, and why do we need them?

You've already been *using* functions since Day 0 — every time you wrote `print(...)`, `len(...)`, or `range(...)`, you were calling a function that Python already built for you. A **function** is a named, reusable block of instructions. Instead of copy-pasting the same five lines of code every time you need to do something, you write those five lines once, inside a function, give it a name, and then just write that name (followed by parentheses) every time you want to run those instructions again — optionally handing it different input each time.

Here's how you define your own:

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Ana")    # prints: Hello, Ana!  
greet("Bo")     # prints: Hello, Bo!
```

Let's name every part of this, since the vocabulary matters for reading documentation and error messages later:

- `def` is the keyword that starts a function definition.
- `greet` is the function's **name** — you choose this, just like a variable name.
- `name` (inside the parentheses) is a **parameter** — a placeholder for a value the function expects to be given each time it's used. Parameters are only defined here, in the function's own definition.
- `"Ana"` and `"Bo"`, when you actually call the function, are **arguments** — the real, specific values you hand in for each parameter, for that particular call.
- The indented lines below `def greet(name):` are the function's **body** — the instructions it runs each time it's called. Exactly like `if` and `for`, indentation is what tells Python which lines belong inside the function.

- `f"Hello, {name}!"` is an **f-string** (formatted string) — the `f` right before the opening quote lets you drop a variable's value directly into the middle of text using `{curly braces}`. You'll use these constantly; full details on Day 5.

"Calling" a function just means writing its name followed by parentheses (with any arguments inside), which tells Python "go run that function's body right now, using these specific argument values."

``return`` — sending a value back out of a function

`print(...)` above just displays something and gives nothing back to use later. Often, you want a function to **compute** something and hand the result back to whatever called it, so you can store it in a variable or use it in further calculations. That's what `return` does:

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)  
print(result)    # 7
```

The moment Python hits a `return` statement, it immediately stops running the function and hands that value back to wherever the function was called from. Any code written after a `return` inside the same block will never run — this is worth knowing because it explains a certain category of "why isn't this line doing anything?" bug.

If a function never hits a `return` statement (like `greet` above, which only `prints`), calling it still technically produces a value — Python automatically gives you back `None` (the "nothing here" value from Day 1) if you try to use the result of a function that never explicitly returned anything:

```
def greet(name):  
    print(f"Hello, {name}!")  
  
output = greet("Ana")    # this DOES print "Hello, Ana!" (the print() call still runs)  
print(output)            # None -- greet() never used `return`, so its "result" is None
```

Default parameter values — making an argument optional

You can give a parameter a default value, which is used automatically if the caller doesn't provide their own:

```
def greet(name, greeting="Hello"):  
    print(f"{greeting}, {name}!")  
  
greet("Ana")                # Hello, Ana!           -- greeting uses its default  
greet("Bo", "Good morning") # Good morning, Bo!    -- greeting overridden explicitly
```

Any parameter with a default value **must come after** all parameters without one — Python requires this ordering so it can always figure out, unambiguously, which argument belongs to which parameter.

A genuinely dangerous trap: mutable default values

This next part is one of the most commonly-cited "gotchas" in all of Python, and it's worth understanding **why** it happens, not just memorizing the fix, because the underlying reason connects directly back to Day 1's mutability lesson.

```
def add_item(item, bucket=[]):    # DANGER -- do not do this
```

```

    bucket.append(item)
    return bucket

print(add_item("a"))    # ['a']
print(add_item("b"))    # ['a', 'b'] <- probably NOT what you expected! You'd expect ['b']

```

Why does this happen? Default values are created exactly **once** — at the moment Python reads the `def` line and creates the function — not fresh on every single call. So `bucket=[]` creates exactly one list object, one single time, and every call that doesn't supply its own `bucket` argument reuses that **same** list. Because lists are mutable (Day 1), `.append(...)` from the first call permanently changes that one shared list, and the second call sees the leftover change from the first.

The fix, which you should treat as a standard pattern to reach for automatically whenever you need a mutable default:

```

def add_item(item, bucket=None):
    if bucket is None:                # note: `is None`, not `== None` -- the idiomatic way to check
        bucket = []                  # a BRAND NEW empty list, created fresh on THIS call only
    bucket.append(item)
    return bucket

```

Now every call that doesn't supply its own bucket gets its own fresh, independent list, created new each time the function actually runs.

`\*args` and `\*\*kwargs` — accepting a flexible number of arguments

Sometimes you don't know in advance exactly how many arguments a function will be called with. `*args` collects any extra positional arguments (arguments given without a name) into a tuple; `**kwargs` collects any extra keyword arguments (arguments given as `name=value`) into a dict. (`args` and `kwargs` are just conventional names — the `*` and `**` are what actually matter.)

```

def describe(name, *tags, active=True, **extra):
    print(name, tags, active, extra)

describe("server1", "prod", "web", region="us-east", owner="ana")
# prints: server1 ('prod', 'web') True {'region': 'us-east', 'owner': 'ana'}

```

Here, `"server1"` fills the required `name` parameter; `"prod"` and `"web"` (extra values with no name attached) are swept up into `tags` as a tuple; `region="us-east"` and `owner="ana"` (extra named values) are swept up into `extra` as a dict. You'll see `*args`, `**kwargs` constantly in real-world code, especially any time one function needs to "forward" whatever it was given along to another function without needing to know its exact signature in advance — this becomes especially important once you meet decorators on Day 10.

Scope — which parts of your code can "see" which variables

Scope refers to which parts of your program a given variable name is visible from. A variable created **inside** a function only exists inside that function — it's invisible to code outside it, and it's created fresh, then thrown away, every single time the function is called:

```
def calculate():
    result = 42
    return result

calculate()
print(result)    # NameError: name 'result' is not defined
```

`result` only exists during the time `calculate()` is actually running — once it returns, `result` is gone. This is genuinely useful: it means the internal, working variables of one function can never accidentally clash with the internal variables of another function, even if they happen to share the same name.

Python looks up a name using a specific search order, often abbreviated **LEGB**: **L**ocal (inside the current function) → **E**nclosing (any function this one is nested inside) → **G**lobal (the top level of your file) → **B**uilt-in (Python's own built-in names, like `print` or `len`). Python checks each level in that order and uses the first match it finds.

```
message = "global"           # a GLOBAL variable -- lives at the top level of the file

def show_message():
    message = "local"        # a LOCAL variable -- only exists inside show_message
    print(message)           # "local" -- found immediately in the Local scope, search stops there

show_message()
print(message)               # "global" -- unaffected; the function's `message` was a totally sep
```

`global` — explicitly modifying a variable from outside a function

By default, if you try to *assign* to a variable inside a function, Python assumes you mean a brand new local variable — even if a global variable of the same name already exists — which leads to a specific, common error:

```
count = 0

def increment():
    count = count + 1    # UnboundLocalError!

increment()
```

This fails because Python sees you assigning to `count` somewhere inside `increment`, decides `count` must be local to this function, and then — reading left to right — tries to read `count`'s current value before it's ever been given one locally. To genuinely modify the outer, global `count`, you must say so explicitly with the `global` keyword:

```
count = 0

def increment():
    global count
    count = count + 1

increment()
print(count)    # 1
```

In real, professional code, relying on `global` is usually considered a sign to reconsider your design — passing values in as parameters and getting results back via `return` (as you did earlier in this lesson) is almost always clearer and less error-prone than quietly mutating some shared global variable from inside a function. You're learning `global` today so you can recognize it and understand *why* the error happens without it — not

because you should reach for it often.

Closures — a function that remembers where it came from

This is one of the more mind-bending ideas in today's lesson the first time you see it, so don't worry if it takes a second read. A function defined *inside* another function can "remember" the variables from the outer function, even after the outer function has already finished running:

```
def make_multiplier(factor):
    def multiply(n):
        return n * factor    # `factor` comes from the ENCLOSING function, not from `multiply` itself
    return multiply          # we return the INNER function itself, not the result of calling it

double = make_multiplier(2)    # factor is 2 for this particular "version" of multiply
triple = make_multiplier(3)    # factor is 3 for THIS version -- entirely separate from double's

print(double(5))    # 10
print(triple(5))    # 15
```

`multiply` is called a **closure**: it "closes over" the variable `factor` from the function it was created inside, and keeps its own private copy of that value alive for as long as the closure itself exists — even though `make_multiplier` finished running and returned long ago. Each separate call to `make_multiplier` creates a completely independent `factor`, which is why `double` and `triple` don't interfere with each other at all, despite being built from the exact same inner function definition.

You'll rely on this exact mechanism to understand decorators on Day 10 — it's worth re-reading this section then if it doesn't fully click yet today.

Functions are values too — you can pass them around like any other data

This is a genuinely important idea: in Python, a function is just another kind of object (remember, from Day 1: everything in Python is an object) — which means you can store it in a variable, put it inside a list or dict, and pass it into other functions as an argument, exactly like a number or a string.

```
def square(x):
    return x * x

operation = square    # note: NO parentheses -- we're referring to the function ITSELF, not calling it
print(operation(5))    # 25 -- calling it through the new name works identically

def apply_twice(func, value):
    return func(func(value))    # calls whatever function was passed in, twice in a row

print(apply_twice(square, 3))    # 81 (square(3) is 9, then square(9) is 81)
```

`lambda` gives you a way to create a small, unnamed ("anonymous") function inline, useful for quick one-off cases, most often as an argument to another function:

```
words = ["banana", "kiwi", "fig"]
words.sort(key=lambda word: len(word))    # sort by each word's LENGTH instead of alphabetically
print(words)    # ['fig', 'kiwi', 'banana']
```

`lambda word: len(word)` is a tiny, unnamed function equivalent to `def get_length(word): return len(word)`. Reach for `lambda` only when the logic is genuinely a single simple expression — anything more involved should be a normal, named `def` function instead, which is easier to read and to debug.

Exercises

Open `exercises.py`, fill in each `# TODO`, then run `python exercises.py` and read the PASS/FAIL results.